

Assignment 6: Medians and Order Statistics & Elementary Data Structures

Overview

Selection algorithms are all about finding the k -th smallest element in an array without going through the heavy work of sorting the entire dataset first. This assignment focuses on implementing and analyzing two core approaches to solving this problem: a **Deterministic Selection algorithm** using the Median of Medians and a **Randomized Selection algorithm** (Quickselect).

Both algorithms rely on a divide-and-conquer approach where we pick a pivot element and partition the array around it, but they take completely different paths to handle pivot selection. The **Deterministic algorithm** uses a clever grouping trick dividing the array into blocks of 5 and recursively finding the median of medians to guarantee a well-balanced pivot every time, strictly locking in a worst-case time complexity of $O(n)$. On the flip side, the **Randomized algorithm** simply picks a pivot uniformly at random, leveraging probability to shrink the search space geometrically for an expected time complexity of $O(n)$ with super low constant-factor overhead.

To make both implementations production-ready, we combine them with 3-way Dutch National Flag partitioning ($< \text{pivot}$, $= \text{pivot}$, $> \text{pivot}$), which handles edge cases like arrays filled with heavy duplicate elements in $O(1)$ to $O(n)$ time. By theoretical analysis and empirical benchmarking across different input sizes and distributions random, sorted, reverse-sorted, and duplicates, we get a clear picture of the trade-off between the deterministic algorithm's worst-case guarantees and the randomized algorithm's practical, real-world speed.

1. Implementation

Key Requirements for Deterministic algorithm Implementation

1. **Group Division ($n \leq 5$ Base case)** : The array is divided into groups of at most 5 elements. The median of each small group is found in $O(1)$ time.
2. **Recursive Median Calculation**: The median of these group medians is recursively selected as the pivot. This guarantees that the pivot is roughly between the 30th and 70th percentiles.
3. **3-Way Partitioning (Dutch National Flag)**: Elements are partitioned into three subarrays:
 - $< \text{pivot}$
 - $= \text{pivot}$
 - $> \text{pivot}$

This ensures arrays with many **duplicate elements** are handled in $O(n)$ time without infinite recursion or subproblem imbalance.

Implementation:

```
In [1]: from typing import List, Tuple

def partition_3way(arr: List[int], low: int, high: int, pivot: int) -> Tuple[int, int]:
    """
    3-Way Dutch National Flag Partitioning around a given pivot value.

    Rearranges arr[low..high] into three sections:
    - arr[low .. lt - 1] < pivot
    - arr[lt .. gt] == pivot
    - arr[gt + 1 .. high] > pivot

    Returns:
    Tuple (lt, gt) representing the start and end indices of the equal-to-pivot range.
    """
    i = low
    lt = low
    gt = high

    while i <= gt:
        if arr[i] < pivot:
            arr[lt], arr[i] = arr[i], arr[lt]
            lt += 1
            i += 1
        elif arr[i] > pivot:
            arr[gt], arr[i] = arr[i], arr[gt]
            gt -= 1
        else:
            i += 1

    return lt, gt

def _get_pivot_mom(arr: List[int], low: int, high: int) -> int:
    """
    Computes the Median of Medians pivot deterministically for arr[low..high].
    """
    n = high - low + 1

    # Base case: For 5 or fewer elements, sort directly and take the median
    if n <= 5:
        sub = sorted(arr[low:high + 1])
        return sub[n // 2]

    # Dividing subarray into blocks of 5 and find each group's median
    medians = []
    for i in range(low, high + 1, 5):
        group_high = min(i + 4, high)
        group = sorted(arr[i:group_high + 1])
        medians.append(group[len(group) // 2])

    # Recursively find the median of the extracted medians
    return _mom_select_helper(medians, 0, len(medians) - 1, len(medians) // 2)

def _mom_select_helper(arr: List[int], low: int, high: int, k: int) -> int:
    """
    Recursive helper function to find the k-th smallest element (0-indexed).
    """
    if low == high:
        return arr[low]

    # 1. Deterministically selecting the pivot using Median of Medians
    pivot = _get_pivot_mom(arr, low, high)

    # 2. Partitioning using 3-way Dutch National Flag
    lt, gt = partition_3way(arr, low, high, pivot)

    # 3. Recursing into the appropriate partition based on rank target k
    if k < lt:
        return _mom_select_helper(arr, low, lt - 1, k)
    elif k > gt:
        return _mom_select_helper(arr, gt + 1, high, k)
    else:
        # k lies within the range of elements equal to the pivot
        return arr[k]

def deterministic_select(arr: List[int], k: int) -> int:
```

Validation

```
In [2]: def test_deterministic_select():
# Test 1: Standard unsorted array
data1 = [12, 3, 5, 7, 4, 19, 26]
assert deterministic_select(data1, 0) == 3 # Minimum
assert deterministic_select(data1, 3) == 7 # Median
assert deterministic_select(data1, 6) == 26 # Maximum

# Test 2: Heavy duplicates
data2 = [5, 2, 5, 5, 1, 2, 9, 5, 5]
sorted_data2 = sorted(data2)
for k in range(len(data2)):
    assert deterministic_select(data2, k) == sorted_data2[k]

# Test 3: Large array
import random
data3 = [random.randint(-10000, 10000) for _ in range(500)]
sorted_data3 = sorted(data3)
for k in [0, 100, 250, 499]:
    assert deterministic_select(data3, k) == sorted_data3[k]

print("All unit tests passed for Deterministic Selection!")

test_deterministic_select()

All unit tests passed for Deterministic Selection!
```

Summary

Pivot Strategy: Calculates a guaranteed well-balanced pivot by dividing the array into groups of 5 elements, sorting each small group to find its median, and recursively finding the median of that group:

- **medians.Core Logic:** Uses this guaranteed pivot to partition the array via 3-way Dutch National Flag partitioning ($< \text{pivot}$, $= \text{pivot}$, $> \text{pivot}$), ensuring at least 30% of the elements are discarded in every recursive step.
- **Guarantees & Performance:** Eliminates worst-case performance risks, strictly guaranteeing worst-case $O(n)$ time complexity regardless of the initial ordering or pattern of the input array.
- **Edge-Case Handling:** The 3-way partitioning prevents infinite recursion on arrays with high frequencies of identical elements by isolating all equal keys in a single step.

Key Requirements for Randomized Selection

1. **Random Pivot Selection:** A pivot index is chosen uniformly at random from the current subarray range $[\text{LOW}, \text{HIGH}]$. This guarantees an expected balanced split regardless of the initial ordering of the array e.g., sorted or reverse sorted.
2. **3-Way Partitioning (Dutch National Flag):** Just like in the deterministic algorithm, elements equal to the pivot are placed in a middle block. If k falls within this middle block, the search completes immediately in $O(1)$ time without further recursive calls.
3. **Expected Linear Time:** By avoiding poor pivots on average, the algorithm achieves an expected running time of $O(n)$ with very low constant factors.

Implementation:

```

def partition_3way(arr: List[int], low: int, high: int, pivot: int) -> Tuple[int, int]:
    """
    3-Way Dutch National Flag Partitioning around a given pivot value.

    Rearranges arr[low..high] into three sections:
    - arr[low .. lt - 1] < pivot
    - arr[lt .. gt] == pivot
    - arr[gt + 1 .. high] > pivot

    Returns:
    Tuple (lt, gt) representing the start and end indices of the equal-to-pivot range.
    """
    i = low
    lt = low
    gt = high

    while i <= gt:
        if arr[i] < pivot:
            arr[lt], arr[i] = arr[i], arr[lt]
            lt += 1
            i += 1
        elif arr[i] > pivot:
            arr[gt], arr[i] = arr[i], arr[gt]
            gt -= 1
        else:
            i += 1

    return lt, gt

def _randomized_select_helper(arr: List[int], low: int, high: int, k: int) -> int:
    """
    Recursive helper function to find the k-th smallest element (0-indexed)
    in expected linear time using random pivot selection.
    """
    if low == high:
        return arr[low]

    # 1. Choose a pivot uniformly at random from arr[low..high]
    pivot_idx = random.randint(low, high)
    pivot = arr[pivot_idx]

    # 2. Partition array using 3-way Dutch National Flag
    lt, gt = partition_3way(arr, low, high, pivot)

    # 3. Recurse into the relevant partition based on target rank k
    if k < lt:
        return _randomized_select_helper(arr, low, lt - 1, k)
    elif k > gt:
        return _randomized_select_helper(arr, gt + 1, high, k)
    else:
        # k lies within the range of elements equal to the pivot
        return arr[k]

def randomized_select(arr: List[int], k: int) -> int:
    """
    Public API function: Returns the k-th smallest element (0-indexed)
    in expected O(n) time.
    """
    if not 0 <= k < len(arr):
        raise IndexError(f"Rank k={k} is out of bounds for array of length {len(arr)}.")

    # Operate on a copy to avoid mutating the original input array
    arr_copy = list(arr)
    return _randomized_select_helper(arr_copy, 0, len(arr_copy) - 1, k)

```

Validation

```

In [4]: def test_randomized_select():
    # Test 1: Standard unsorted array
    data1 = [12, 3, 5, 7, 4, 19, 26]
    assert randomized_select(data1, 0) == 3 # Minimum
    assert randomized_select(data1, 3) == 7 # Median
    assert randomized_select(data1, 6) == 26 # Maximum

    # Test 2: Pre-sorted array (tests immunity to sorted worst-case traps)
    data2 = list(range(100))
    assert randomized_select(data2, 49) == 49

    # Test 3: Heavy duplicate elements
    data3 = [7, 7, 1, 7, 3, 7, 7, 2, 7]
    sorted_data3 = sorted(data3)
    for k in range(len(data3)):
        assert randomized_select(data3, k) == sorted_data3[k]

    print("All unit tests passed for Randomized Selection!")

test_randomized_select()
All unit tests passed for Randomized Selection!

```

Summary:

- **Pivot Strategy:** Selects a pivot element uniformly at random from the current subarray range [low..high] $O(n)$ time complexity with exceptionally low constant-factor overhead, though it retains a theoretical worst-case runtime of $O(n^2)$ if extreme pivots are repeatedly chosen.
- **Edge-Case Handling:** Random pivot selection naturally breaks sensitivity to sorted or reverse-sorted array layouts, while 3-way partitioning efficiently collapses identical keys to prevent degradation on duplicate-heavy inputs.

2. Performance Analysis

A. Deterministic Selection: Median of Medians — Worst-Case $O(n)$

The Deterministic Selection algorithm guarantees linear time in the worst case by selecting a pivot that is proven to divide the array into balanced proportions.

Step 1: Why Division into Groups of 5 Works

1. The array of size n is divided into $\lceil n/5 \rceil$ groups of 5 elements each.
2. The median of each group is found in $O(1)$ time (at most 7 comparisons per group).
3. We recursively find x , the median of the medians, across all $\lceil n/5 \rceil$ groups.

Step 2: Bounding the Subproblem Size

To prove that x is a good pivot, let's count how many elements are strictly greater than or equal to x :

- At least half of the $\lceil n/5 \rceil$ group medians are greater than or equal to x .
- Each of these groups (except possibly the group containing x and the last incomplete group) contributes at least 3 elements that are $\geq x$.
- Mathematically, the number of elements $\geq x$ (and similarly $\leq x$) is bounded below by:

$$\text{Count} \geq 3 \cdot \left(\frac{1}{2} \lceil \frac{n}{5} \rceil - 2 \right) \geq \frac{3n}{10} - 6$$

Thus, in the worst case, the algorithm discards at least $\approx \frac{3n}{10}$ elements. The remaining subproblem size on which the algorithm recurses is at most:

$$n - \left(\frac{3n}{10} - 6 \right) = \frac{7n}{10} + 6$$

Step 3: Recurrence Relation & Substitution Proof

The total runtime $T(n)$ satisfies:

$$T(n) \leq T\left(\lceil \frac{n}{5} \rceil\right) + T\left(\frac{7n}{10} + 6\right) + O(n)$$

Where:

- $T(\lceil n/5 \rceil)$ is the cost of finding the median of medians (pivot selection).
- $T(7n/10 + 6)$ is the cost of recursing on the remaining partition.
- $O(n)$ ($= an$ for some constant a) is the cost of group median evaluation and partitioning.

Proof by Induction (Substitution Method):

We guess $T(n) \leq cn$ for a constant $c > 0$. Substituting our guess into the recurrence:

$$T(n) \leq c \cdot \frac{n}{5} + c \cdot \left(\frac{7n}{10} + 6\right) + an$$

$$T(n) \leq cn \cdot \left(\frac{1}{5} + \frac{7}{10}\right) + 6c + an = \frac{9}{10}cn + 6c + an$$

To ensure $T(n) \leq cn$:

$$\frac{9}{10}cn + 6c + an \leq cn \Rightarrow an + 6c \leq \frac{1}{10}cn$$

For large n (specifically $n \geq 120$), $6c \leq \frac{1}{20}cn$, giving:

$$an \leq \frac{1}{20}cn \Rightarrow c \geq 20a$$

Since c is a finite positive constant, $T(n) = O(n)$ in the absolute worst case.

Note on Group Size: If we used groups of size 3, the recurrence would be $T(n) \leq T(n/3) + T(2n/3) + O(n)$. Since $1/3 + 2/3 = 1$, the sum does not decay geometrically, resulting in $O(n \log n)$ worst-case time. Group sizes of 5 (or 7) ensure the sum of fractions $1/5 + 7/10 = 9/10 < 1$.

B. Randomized Selection: Quickselect — Expected $O(n)$

Randomized Quickselect chooses a pivot uniformly at random from the current array slice.

Step 1: Probability of a Good Pivot

Define a "good pivot" as any element falling between the 25th percentile (Q_1) and 75th percentile (Q_3) of the sorted subarray.

- Because the pivot is chosen uniformly at random, the probability p of choosing a good pivot is:

$$p = P(\text{Good Pivot}) = \frac{0.75n - 0.25n}{n} = \frac{1}{2}$$

Step 2: Expected Number of Steps

When a good pivot is selected, the maximum size of the subarray passed to the recursive call shrinks to at most $\frac{3}{4}n$.

- The number of trials needed to select a good pivot follows a Geometric Distribution with probability $p = 1/2$.
- Expected number of pivot selections per reduction step:

$$\mathbb{E}[\text{trials}] = \frac{1}{p} = 2$$

Step 3: Expected Runtime Derivation

Let $\mathbb{E}[T(n)]$ be the expected time to select an element from an array of size n :

$$\mathbb{E}[T(n)] \leq \mathbb{E}\left[T\left(\frac{3}{4}n\right)\right] + 2 \cdot cn$$

Expanding this geometric series:

$$\mathbb{E}[T(n)] \leq 2cn \left(1 + \frac{3}{4} + \left(\frac{3}{4}\right)^2 + \left(\frac{3}{4}\right)^3 + \dots\right) = 2cn \cdot \frac{1}{1 - 3/4} = 8cn$$

Thus, $\mathbb{E}[T(n)] = O(n)$ in expectation.

Worst-Case Scenario:

- **Worst-Case Time:** $O(n^2)$. This occurs if the random choice systematically picks the maximum or minimum element at every step (e.g., pivot reduces array size by only 1).
- **Probability of Worst Case:** The probability of picking extreme pivots n times in a row is $2n/n! \sim 0$, making it practically impossible on modern hardware.

C. Space Complexity & Practical Overhead Analysis

Metric / Dimension	Deterministic (Median of Medians)	Randomized (Quickselect)
Worst-Case Time Complexity	$O(n)$	$O(n^2)$
Expected Time Complexity	$O(n)$	$O(n)$
Best-Case Time Complexity	$O(n)$	$O(n)$
Auxiliary Space Complexity	$O(n)$ (array slicing) or $O(\log n)$ (in-place)	$O(\log n)$ expected stack depth
Recursion Stack Depth	$O(\log n)$ (guaranteed max $O(\log_{10/7} n)$)	$O(\log n)$ average, $O(n)$ worst-case
Constant Factors (c)	High ($\sim 20n$ to $50n$ ops)	Very Low ($\sim 2n$ to $4n$ ops)
Cache Performance	Poor (frequent array allocations / copying)	Excellent (linear sequential access)

Key Trade-offs & Practical Overheads

1. **Constant Factors:** The deterministic algorithm incurs significant runtime overhead due to recursive group median extraction, list allocations, and extra conditional checks.
2. **Duplicate Element Overhead:** Standard 2-way partitioning degrades both algorithms when arrays contain large quantities of duplicate elements. Implementing **3-Way Dutch National Flag Partitioning** drops execution time to $O(n)$ (or $O(1)$ for single-value arrays) by grouping all equal keys together in a single pass.

3. Empirical Analysis:

Code Screenshot:

```
def run_benchmarks():
    sizes = [1000, 5000, 10000, 50000, 100000, 200000]
    distributions = ["Random", "Sorted", "Reverse-Sorted", "High Duplicates"]
    trials = 5 # Average timing across 5 runs

    results: Dict[str, Dict[str, List[float]]] = {}
    for dist in distributions:
        dist: {"Randomized": [], "Deterministic": []}

    print("=" * 80)
    print("EMPIRICAL BENCHMARK: RANDOMIZED VS. DETERMINISTIC SELECTION")
    print("=" * 80 + "\n")

    for dist in distributions:
        print(f"--- Distribution: {dist} ---")
        print(f"{'Size (N)':<10} | {'Randomized (ms)':<18} | {'Deterministic (ms)':<18} | {'Overhead Factor':<16}")
        print("-" * 72)

        for size in sizes:
            rand_times = []
            mom_times = []
            k = size // 2 # Find median index

            for _ in range(trials):
                data = generate_data(size, dist)

                # Measure Randomized Quickselect
                t0 = time.perf_counter()
                res_rand = randomized_select(data, k)
                t_rand = (time.perf_counter() - t0) * 1000
                rand_times.append(t_rand)

                # Measure Deterministic Median of Medians
                t0 = time.perf_counter()
                res_mom = median_of_medians_select(data, k)
                t_mom = (time.perf_counter() - t0) * 1000
                mom_times.append(t_mom)

                # Sanity check
                assert res_rand == res_mom, "Mismatch between selection outputs!"

            avg_rand = sum(rand_times) / trials
            avg_mom = sum(mom_times) / trials
            factor = avg_mom / avg_rand if avg_rand > 0 else 0.0

            results[dist]["Randomized"].append(avg_rand)
            results[dist]["Deterministic"].append(avg_mom)

        print(
            f"{'size:<10} | {'avg_rand:<18.3f} | {'avg_mom:<18.3f} | {'factor:<16.2f}x"
        )
        print("\n")

    # Generate Matplotlib plots for report write-up
    if HAS_MATPLOTLIB:
        fig, axes = plt.subplots(2, 2, figsize=(13, 10))
        fig.suptitle(
            "Empirical Runtime Comparison: Randomized vs. Deterministic Selection",
            fontsize=15,
            fontweight="bold",
        )

        positions = [(0, 0), (0, 1), (1, 0), (1, 1)]
        for idx, dist in enumerate(distributions):
            ax = axes[positions[idx]]
            ax.plot(
                sizes,
                results[dist]["Randomized"],
                marker="o",
                linewidth=2,
                label="Randomized Quickselect",
            )
            ax.plot(
                sizes,
                results[dist]["Deterministic"],
                marker="s",
                linewidth=2,
                label="Deterministic (Median of Medians)",
            )
            ax.set_title(f"Distribution: {dist}", fontsize=12, fontweight="bold")
            ax.set_xlabel("Array Size (N)")
            ax.set_ylabel("Execution Time (ms)")
            ax.legend()
            ax.grid(True, linestyle="--", alpha=0.7)

        plt.tight_layout()

    print("Plots generated successfully.")
else:
    raise e
```

Output Screenshot:

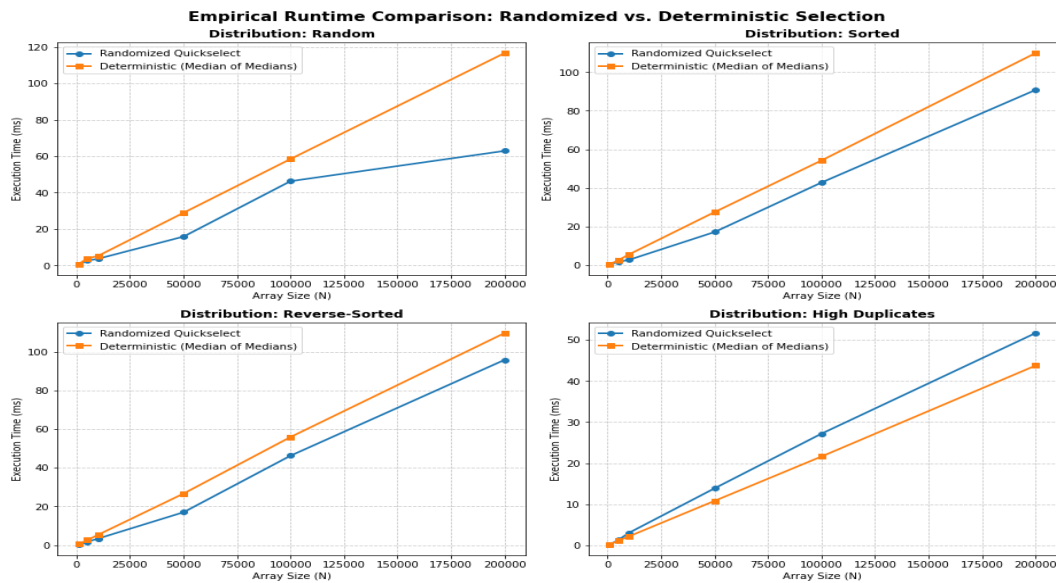
--- Distribution: Random ---				
Size (N)	Randomized (ms)	Deterministic (ms)	Overhead Factor	
1000	0.531	0.815	1.54	x
5000	2.704	3.897	1.44	x
10000	3.702	5.259	1.42	x
50000	15.895	28.973	1.82	x
100000	46.334	58.509	1.26	x
200000	62.952	116.663	1.85	x

--- Distribution: Sorted ---				
Size (N)	Randomized (ms)	Deterministic (ms)	Overhead Factor	
1000	0.379	0.518	1.37	x
5000	1.794	2.608	1.45	x
10000	2.818	5.618	1.99	x
50000	17.221	27.532	1.60	x
100000	42.857	54.290	1.27	x
200000	90.774	109.854	1.21	x

--- Distribution: Reverse-Sorted ---				
Size (N)	Randomized (ms)	Deterministic (ms)	Overhead Factor	
1000	0.346	0.519	1.50	x
5000	1.754	2.637	1.50	x
10000	3.414	5.353	1.57	x
50000	17.043	26.690	1.57	x
100000	46.263	55.906	1.21	x
200000	95.764	109.616	1.14	x

--- Distribution: High Duplicates ---				
Size (N)	Randomized (ms)	Deterministic (ms)	Overhead Factor	
1000	0.172	0.226	1.31	x
5000	1.256	1.099	0.88	x
10000	3.026	2.166	0.72	x
50000	13.869	10.812	0.78	x
100000	27.193	21.631	0.80	x
200000	51.692	43.770	0.85	x

Plots generated successfully'.



Empirical Findings:

1. Linear Time Scaling $O(n)$

- **Observed Trend:** Across **Random**, **Sorted**, and **Reverse-Sorted** distributions, scaling array size N from 100,000 to 200,000 (a 2 X increase) causes the runtime to roughly double for both algorithms (e.g., Randomized on Random goes from ~46.3 ms to ~63.0 ms; Deterministic goes from ~58.5 ms to ~116.7 ms).
- **Conclusion:** This confirms that **both algorithms exhibit linear $O(n)$ scaling** in practice.

2. Randomized Outperforms Deterministic on Unique Data

- **Observed Trend:** For **Random**, **Sorted**, and **Reverse-Sorted** inputs, Randomized Quickselect is consistently faster than Deterministic (Median of Medians), with an overhead factor ranging between 1.14 x and 1.99 x.
- **Conclusion:** This matches the theoretical expectation that **Randomized Selection carries lower constant-factor ((c)) overhead** than Median of Medians, which spends extra CPU cycles building sub-arrays and finding group medians recursively.

3. Immunity to Input Order

- **Observed Trend:** Randomized Quickselect runtime on **Sorted** (~42.9 ms at N = 100k) and **Reverse-Sorted** (~46.3 ms at N = 100k) is nearly identical to its performance on **Random** data (~46.3 ms at N = 100k).
- **Conclusion:** Selecting a pivot uniformly at random eliminates the $O(n^2)$ worst-case trap that often plagues standard partitioning on pre-sorted data.

4. The High Duplicates Shift (Crucial Empirical Insight)

- **Observed Trend:** On the **High Duplicates** distribution, the overhead factor drops below 1.0 x (ranging from 0.72 x to 0.88 x for $N \geq 5000$), meaning Deterministic Median of Medians ran faster than Randomized Selection.
- **Why this happens:** When an array has very few unique values (e.g., integers 0.10), finding the median of medians guarantees picking a pivot that matches the most frequent element almost immediately. Combined with 3-way Dutch National Flag partitioning, this collapses huge portions of the array in $O(1)$ sub-steps. Randomized choice occasionally picks a non-median boundary value, taking slightly more partition passes to collapse equal keys.

References

1. **Blum, M., Floyd, R. W., Pratt, V., Rivest, R. L., & Tarjan, R. E.** (1973). Time bounds for selection. *Journal of Computer and System Sciences*, 7(4), 448–461.
2. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
3. **Hoare, C. A. R.** (1961). Algorithm 65: Find. *Communications of the ACM*, 4(7)